CSC209 Project Report

Erik Dadashyan Kai Tano Bague Joshua Chen

April 6, 2026

1 Project Overview: Quick Splash

Welcome to Quick Splash! A CLI based game similar to Quiplash and Cards Against Humanity, combined with a voting system for the users to vote on their favourite prompt/phrase combos.

This game differs from its reference material in that no single user is responsible for writing the immediate prompt, instead, the host system will choose a random prompt from a locally stored file.

Each client will receive a copy of said prompt, and be given 60 seconds to respond to the prompt accordingly. Many prompts are "fill in the blanks", but some could differ in not having any blanks, or having multiple blanks. The user can use their 256 character response buffer to answer whatever they would like in this section.

The user's response is then sent to the server, who will then concatenate all of the responses together into an array and share that array with all of the other clients so that they can vote on their favourite response. Differing from normal Quiplash as well, we do not anonymize the response, so as to encourage discussion amongst the players regarding each user's answer.

Once all of the clients have voted, the server will process and tally all of the votes, and send the final results back to the client for an undetermined amount of time. This period after the round allows the users to laugh at the responses and gives them sufficient time to discuss. The host user can continue to the next round at any time.

Once all rounds have been played, the host will then tally up the total number of wins each user attained, and crown the game winner accordingly.

The example prompts are sourced from the Cards Against Humanity Family Pack, which is licensed under Creative Commons.

2 Build Instructions

As a server, you should `cd` to the project directory and run the command `make server`, and then run `./server`.

As a client/user, you should also `cd` to the project directory, then run the command `make client`, and finally run `./client`.

Alternatively, you could also `make all` and then run either `./server` or `./client`.

3 Architecture Diagram

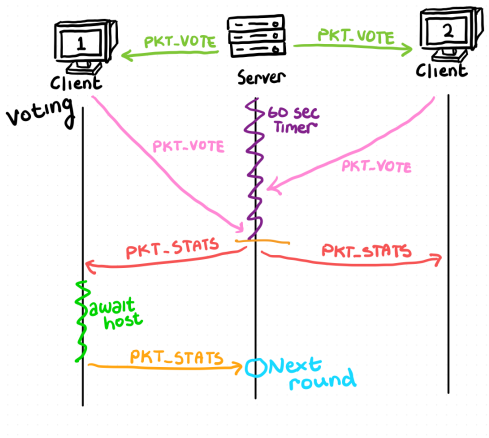

Figure 1: Architecture diagram for starting the game

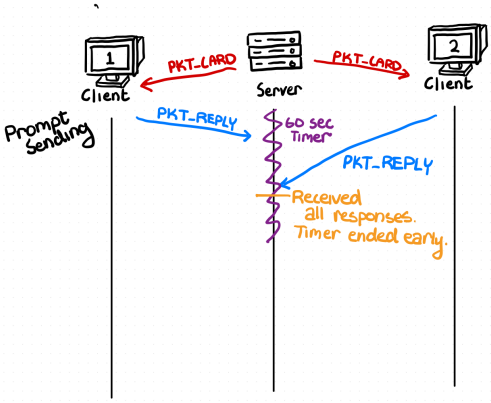

Figure 2: Architecture diagram for prompt sending

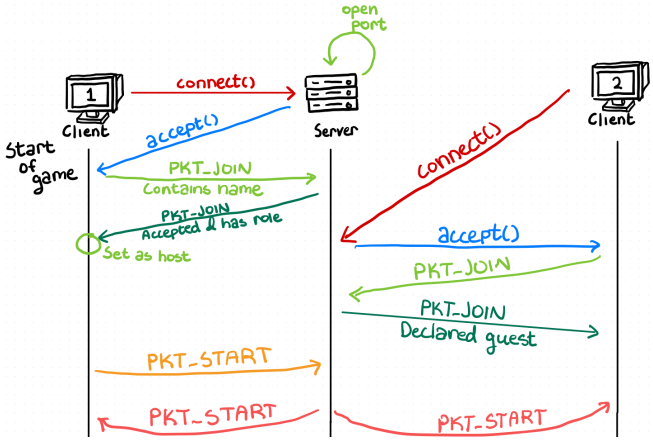

Figure 3: Architecture diagram for voting

4 Communication Protocol

The table below is a template for documenting each protocol message type.

Message Type	**Sender→Receiver**	**Semantics and Receiver Invariants**
PKT_JOIN	Client ↔ Server	Client creates PKT_JOIN with client's chosen name and sends to server. Server responds with PKT_JOIN confirming receipt, and also indicating the player's role on the server (guest or host). The role is decided by join order.
PKT_CARD	Server → Client	Server creates cards which are converted to packets of exactly the length of the prompt (and necessary overhead). This is packed into the stream, received and handled accordingly by the player instances.
PKT_REPLY	Client → Server	Meaning: Client sends to the server their response to the prompt. The server will then save the response (by duplicating it on the server instance) to its local version of the Card struct, which contains an array of character pointers to each user's response.
PKT_VOTE	Server ↔ Client	The server sends the original prompt with each player's response. Each client is in charge of displaying the vote and handling vote input. Clients will respond to the server with a PKT_VOTE as well, indicating the client's choice. This is verified on both ends and tallied on the serverside.
PKT_STATS	Server → Client	The host sends the client a Packet containing the results of the vote or of the entire game.
PKT_QUIT AND PKT_PLR_DC	Client ↔ Server	A client has disconnected from the server and its signal has been detected by the server. Alternatively, the client disconnected via a SIGINT which is caught and results in a PKT_QUIT packet being sent. The server holds this signal until the next memory safe place to do garbage cleanup on the removed user. Then, it sends the disconnect to all remaining clients to update their local player counts.
PKT_GAME_END	Server → Client	The game is over, the host sends a signal to all clients to end the game.

Table 1: Communication protocol message specification template.

5 Concurrency Model

We achieve concurrency by always having the server be listening to all clients for the entire duration of the lobby loop and game loop. This is done through the `s_listen(int max_time)` function, which constructs an `fd_set` consisting of all of the players file descriptors.

```
fd_set fds;
FD_ZERO(&fds);
for (int i = 0; i < LOBBY_SIZE; i++) {
    if (pending[i] && players[i].fd > 2) { // only listen to players who are pending and
        also not disconnected
        FD_SET(players[i].fd, &fds);
        if (players[i].fd > high) {
            high = players[i].fd;
        }
    }
}

if (high == 0) {
    return TIMEOUT;
}
```

It notes the highest individual file descriptor which has not already been read from in each iteration of the while loop. It also implements a timeval struct as a synchronous IO multiplexer, so as to achieve a `max_time` value after which the server will time out and send a packet to all clients informing them that time has run out for that given prompt/vote/relevant game state.

```
int t = time(NULL) - start;
if (t >= max_time) {
    return TIMEOUT;
}
struct timeval tl;
tl.tv_sec = max_time - t;
tl.tv_usec = 0;
int listen = select(high + 1, &fds, NULL, NULL, &tl);
if (listen > 0) {
    for (int i = 0; i < LOBBY_SIZE; i++) {
        if (pending[i] && players[i].fd > 2 && FD_ISSET(players[i].fd, &fds)) {
            response ret = s_read(&players[i]);
```

Beyond that, we avoid blocking on a single client by ensuring that once the client sends a message, they are not expected until the next call of this function call to send in a response. We pass the timeval struct as the IO multiplexer for the actual select value.

The actual reading of the packet data is achieved using the `s_read(Player *p)` function, which grabs the file descriptor from the appropriate player struct, and as such, in our case upon receiving a select value indicative of a player beginning to send their Packet Header. This function will set the state of the player on the server side to be such that we are awaiting a header. We read what we can from the client, and return to the loop. Once we have received the full header, we set the player's state to Payload, wherein we begin listening for the number of bytes stated by the

client in the header. The header also contains the type of the packet, and as such, we attach the PacketHeader to the actual Packet it is describing so that we can use it later on. Once a full packet is achieved, we move the packet from the buffer to the active data for the client.

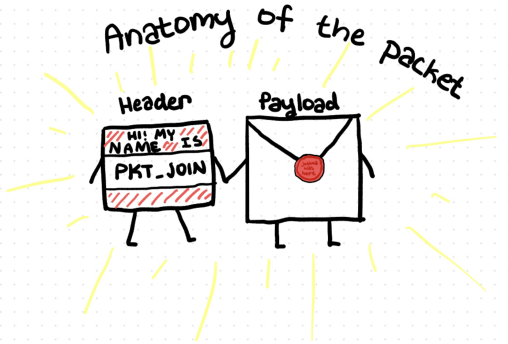

Figure 4: Diagram of a packet

In the event a client does not send a full header or anything which would otherwise cause a block (i.e. a disconnect), we have already accounted for the fact that we may not receive the full PacketHeader or Payload in one given transmission and as such, we have appropriate buffers for both.

6 Error Handling and Robustness

Client disconnects from the game: there are two different types of disconnections that the server has to handle, a graceful one and a non-graceful one. The graceful occurs when the client sends a signal that is caught (i.e. SIGINT), in this case the client will send the server a packet that will signify the client's disconnect and will default out their responses, where in the voting phase their prompt response would be set to "I ragequit" and their vote response won't count in the round's vote results. This ensures that the `PLAYER_COUNT` is not mutated mid round, as mutating this value could interfere with the data structures due to the fact that the responses array holds `PLAYER_COUNT response` structures, if we reduce by 1, we may remove access to the last player in the array which may not always be the player who disconnects. Non-graceful disconnects are handled in a similar way but instead the server will timeout at every response phase and will set their response to NULL but their vote will not count. After the round is over, when it is safe to do so, the server will enter a brief garbage collection phase wherein it will mark all disconnected instances which were previously marked into being a zombie with file descriptor -1 into properly being disconnected users. It then also sends the appropriate packets to all players so as to tell each user that the disconnect occurred and to increment their internal player count variables accordingly.

Malformed/Missing/Incomplete Packet: when we receive an incorrect/expected Packet on the server, it will likely be caught in one of several places, the most likely of which would be when the server attempts to read their PacketHeader and it violates the predetermined constraints. The PacketHeader contains a specific enum for the type of the packet, as well as the length. The length is stored as an `uint16_t` so as to limit the damage from a potential malicious attack, or just avoid the host awaiting a negative value.

If it manages to successfully send a full header which contains the expected values and has a reasonably expected type, then we no longer have a malformed Packet, unless they fail to send an appropriate amount of data, wherein that client's response for that round/task was not fully received and the Server will treat it as an incomplete/unreceived packet and clear the buffers accordingly in preparation for the next round if there is anything inside regardless of completeness.

Players providing incorrect inputs: player input is checked right after receiving it (`fgets` returns a newline terminated string) and checked thoroughly. The only places in which user input could be a problem are for ports/server ip addresses, but we just have to feed what the user typed in into the socket connection and it will either work or not work. Since we are printing strings (user responses for cards or usernames), we have to always make sure their input remains in `char*` buffers and that we do not have any `printf(buffer)` as this could cause security issues. All of our text input is printed using `printf("%s", buffer)` or something similar to ensure it is only treated as a sequence of characters.

7 Team Contributions

Joshua Responsible for communication protocols for client/host connection and packets. Also responsible for the implementation of the basic client and server systems, including init.c and comms.c iterations for both ends. Provided general assistance in other categories for implementation of actual functionality. Introduced length prefixed variable length packets for the communication, modelled after the needs of the voting system.

Erik Responsible for core game logic by implementing gamestructs.c and gamestates.c. Designing and implementing game entities, such cards.c and others in gamestructs.h. Responsible for storing player responsse data and player stats within entities. Lastly managed how data was compiled to send over to clients.

Kai Tano Responsible for terminal input/display and inner game logic. I worked on the voting state in gamestates.c including sending votes, gathering votes, sending voting results and waiting for host to start next roung. Contributed in handling user input and terminal output and making sure they do not result in client timeouts from the server. I provided some help in comms.c to improve handling of disconnected clients while the server is in the middle of a game state logic (like waiting for clients to send their responses/votes).